INTERRUPT INTERACTION WITH FREE RTOS TASK

Assignment 05

CSE323 Advanced Embedded Systems

Table of Contents

1.0 Executive Objective 2
2.0 System Architecture & Execution Flow 2
2.1 Task & ISR Hierarchy 2
3.0 Screenshot Verification & Sequential Analysis 2
3.1 Experimental Result Summary 11
4.0 Interpretation & Engineering Trade-offs 11
5.0 Conclusion 11

Figure 1 Initial Debug Setup 3
Figure 2 First ISR Hit (Pre-Increment) 4
Figure 3 First ISR Hit (Post-Increment) 5
Figure 4 Handler Task Preemption (Pre-Processing) 6
Figure 5 Handler Task (Post-Processing) 7
Figure 6 ISR Overlap Condition 8
Figure 7 Second ISR Event Registration 9
Figure 8 Definitive Proof of Event Loss 10

1.0 Executive Objective

The primary objective of this report is to empirically demonstrate the architectural limitations of binary semaphores within a FreeRTOS environment, specifically during high-frequency interrupt bursts. Because a binary semaphore is strictly a dual-state mechanism (Empty or Full), it cannot queue multiple pending tokens. This assignment proves that if subsequent hardware or software interrupts trigger while the deferred processing task is still executing, those additional synchronization events will be irrevocably lost.

2.0 System Architecture & Execution Flow

To reliably recreate an interrupt overload scenario, the system was decoupled into three primary components: an automated stimulus generator, an Interrupt Service Routine (ISR), and an artificially constrained handler task.

2.1 Task & ISR Hierarchy

- **vTriggerTask() (The Stimulus):** Engineered to generate a rapid, automated burst of three software-triggered interrupts utilizing NVIC_SW_TRIG_R (specifically interrupt 30). This approach eliminates the inconsistencies of physical button debouncing and guarantees a deterministic overload condition.
- **GPIOF_Handler() (The Interceptor):** Represents the asynchronous interrupt source. Upon execution, it:
    - Clears the active interrupt flag.
    - Increments the gInterruptsFired diagnostic counter.
    - Toggles an onboard LED for physical state indication.
    - Attempts to yield a token via xSemaphoreGiveFromISR().
    - Dynamically increments either gSemaphoreAccepted or gDroppedInterrupts based on the success or failure of the semaphore give operation.
- **vHandlerTask() (The Bottleneck):** Acts as the deferred processing mechanism. Upon acquiring the semaphore token via xSemaphoreTake(), it asserts a gHandlerBusy = 1 flag, increments gTasksProcessed, and intentionally stalls execution using vTaskDelay(1200). This simulated computational delay guarantees the task remains engaged while the remaining interrupts from the burst arrive.

3.0 Screenshot Verification & Sequential Analysis

To capture undeniable proof of the dropped interrupts occurring at microprocessor speeds, the Keil IDE's debugging environment and watch variables were utilized to freeze the system state across the execution sequence. All captured states were verified against the theoretical execution model.

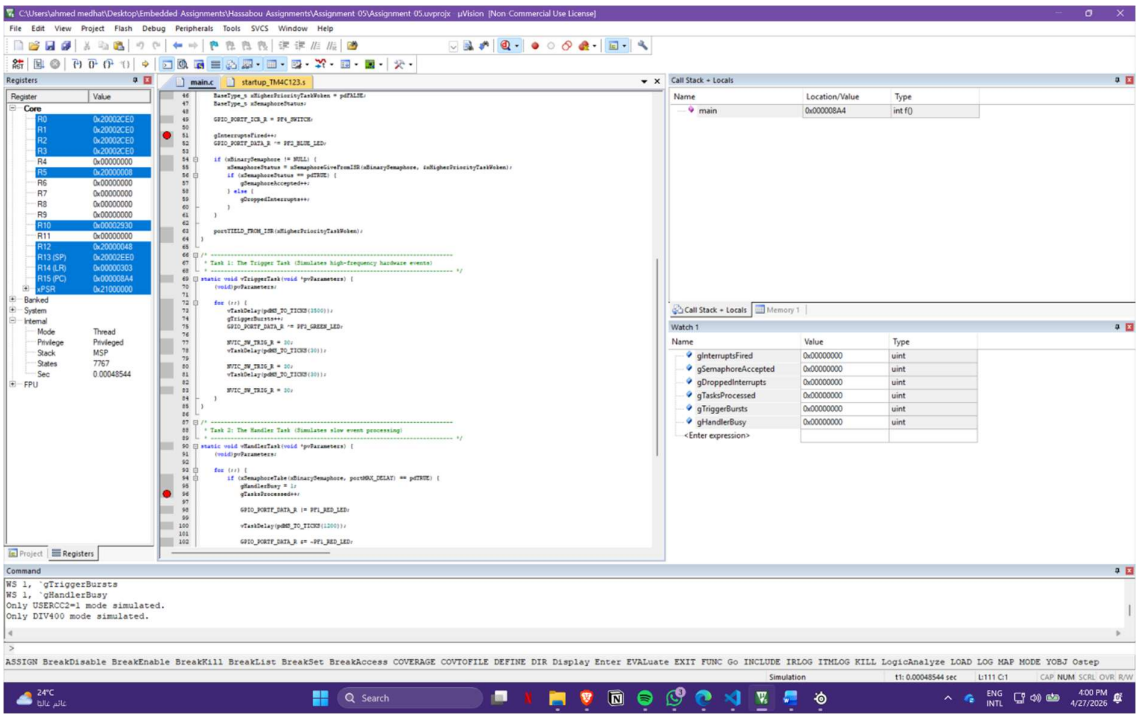

Figure 1 Initial Debug Setup

This capture illustrates the initial debug state prior to the execution of the primary interrupt burst. Both breakpoint anchors are visible, and all diagnostic watch variables remain at zero. This establishes a valid, uncontaminated baseline state for the experiment.

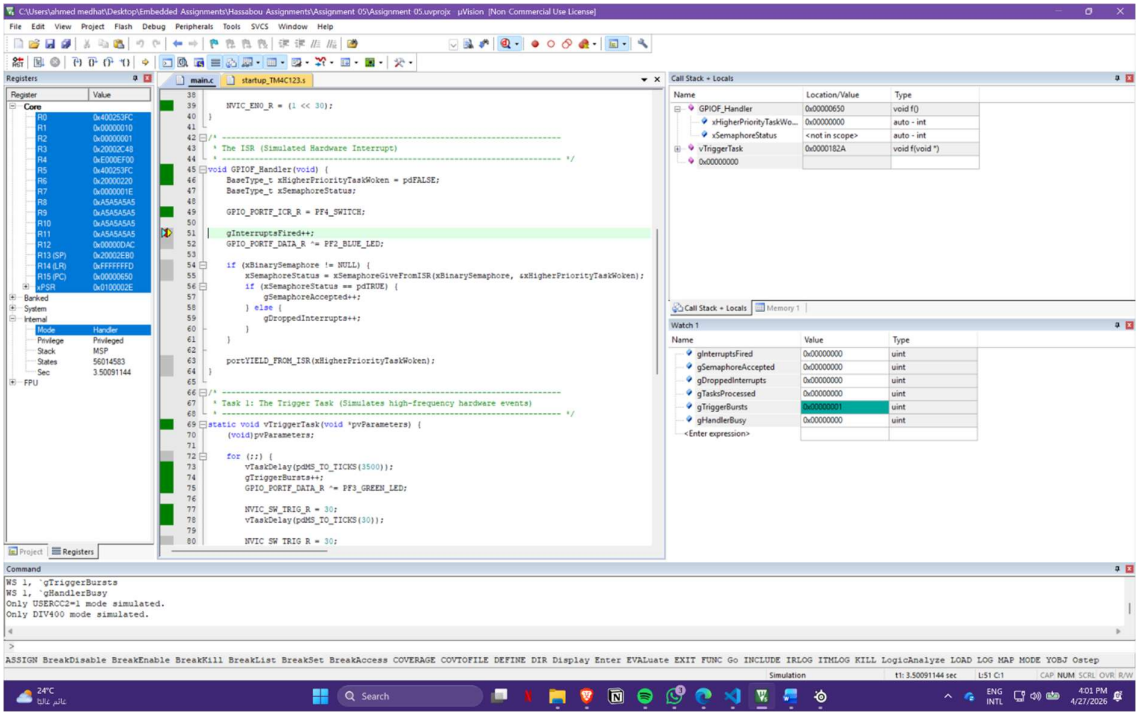

Figure 2 First ISR Hit (Pre-Increment)

Execution halts within GPIOF_Handler() exactly at the breakpoint preceding gInterruptsFired++;.

- gInterruptsFired = 0
- gTriggerBursts = 1

This confirms the system successfully intercepted the initial burst trigger before modifying internal tracking counters.

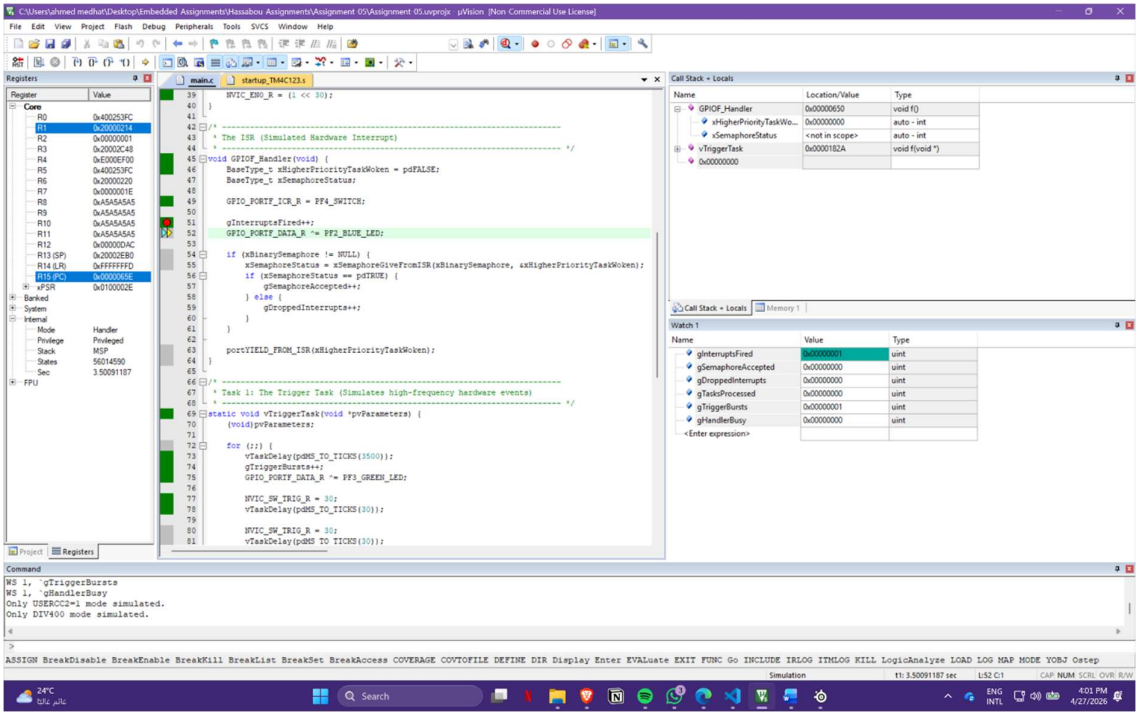

Figure 3 First ISR Hit (Post-Increment)

Following a single execution step, the counter registers the event:

- gInterruptsFired = 1

This confirms that the first interrupt in the burst was successfully generated and processed by the ISR logic.

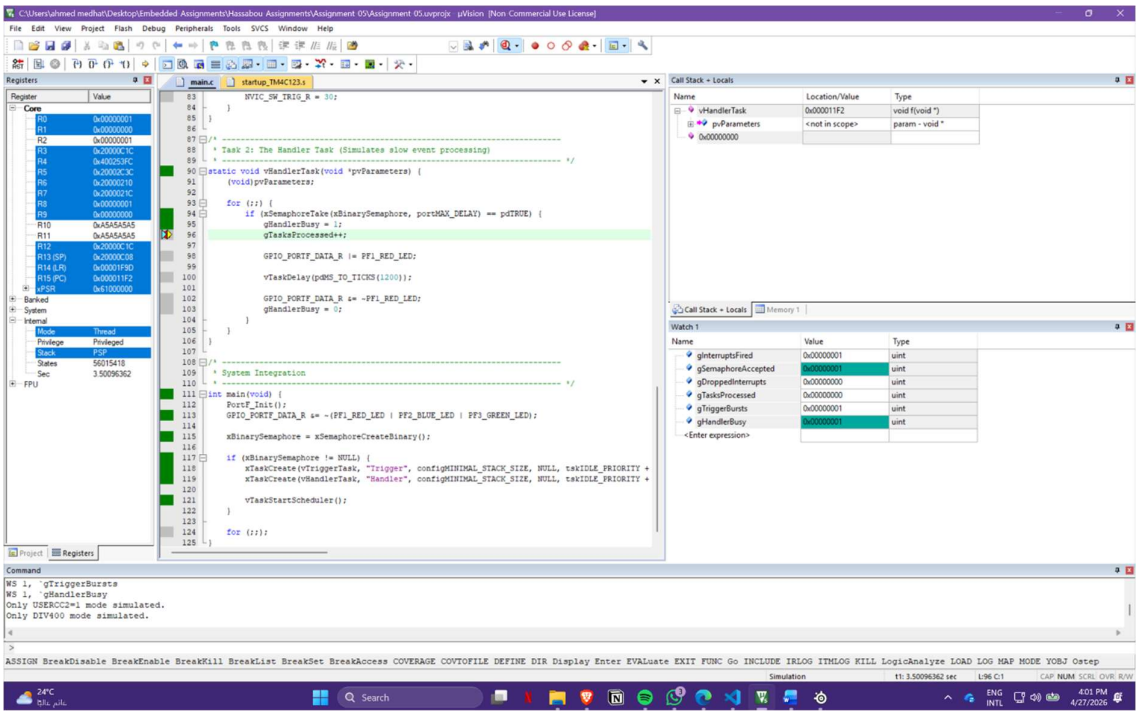

Figure 4 Handler Task Preemption (Pre-Processing)

The scheduler has awakened vHandlerTask(), halting at gTasksProcessed++;.

- gInterruptsFired = 1
- gSemaphoreAccepted = 1
- gTasksProcessed = 0
- gHandlerBusy = 1

This perfectly matches the theoretical model: the first interrupt token was accepted by the binary semaphore, causing the handler task to wake, preempt the idle state, and assert its busy flag prior to processing.

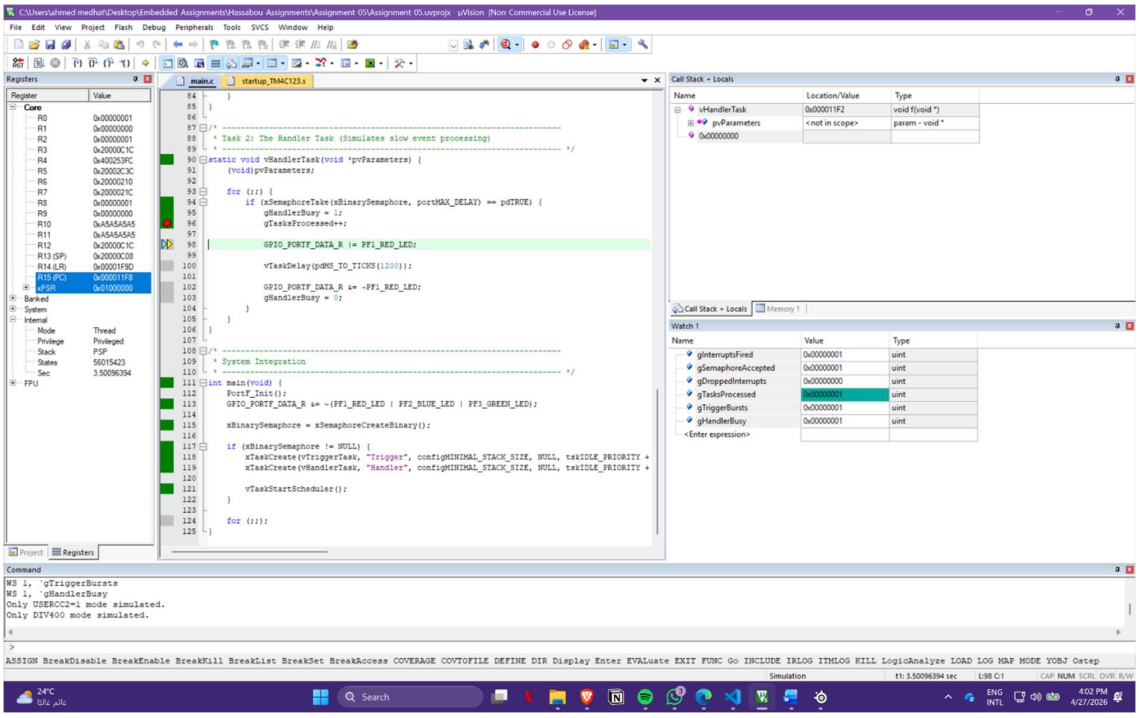

Figure 5 Handler Task (Post-Processing)

Stepping forward within the handler task updates the system state:

- gTasksProcessed = 1
- gHandlerBusy = 1

This proves the first interrupt event has been formally processed by the application layer while the task remains in its active (busy) cycle.

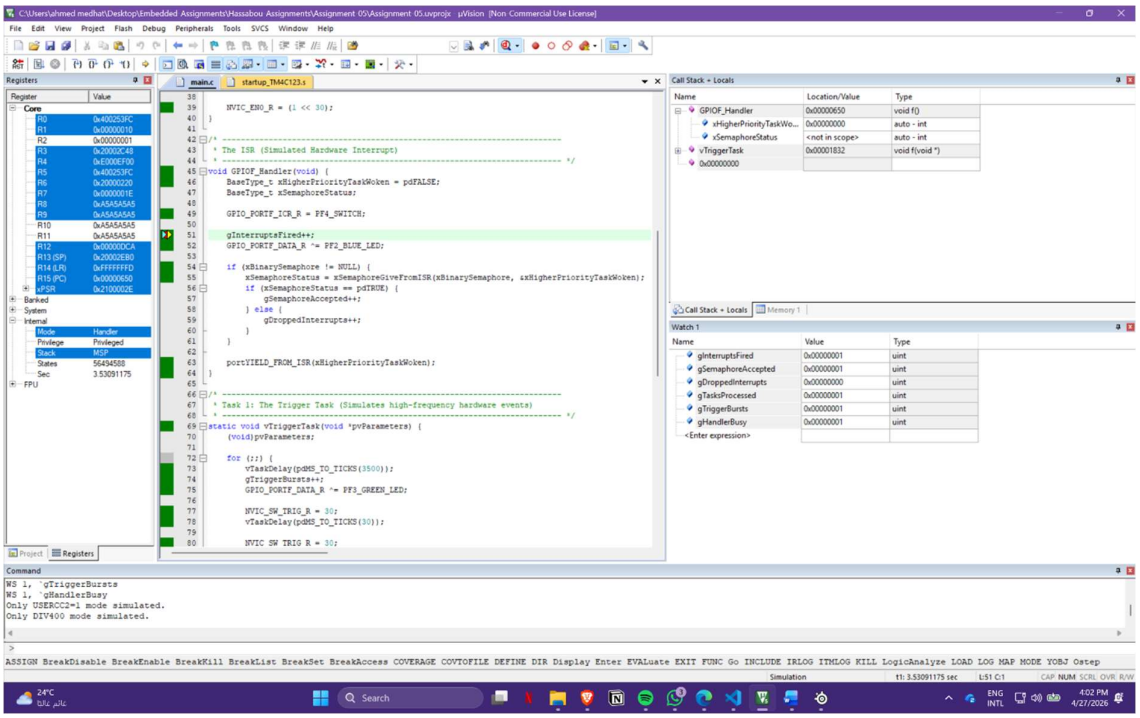

Figure 6 ISR Overlap Condition

Execution is forced back into GPIOF_Handler() due to a secondary hardware trigger, while the baseline states remain:

- gTasksProcessed = 1
- gHandlerBusy = 1
- gSemaphoreAccepted = 1
- gDroppedInterrupts = 0

This is a critical intermediate state. It proves that a subsequent interrupt successfully preempted the system *while* the handler task was still locked in its processing delay.

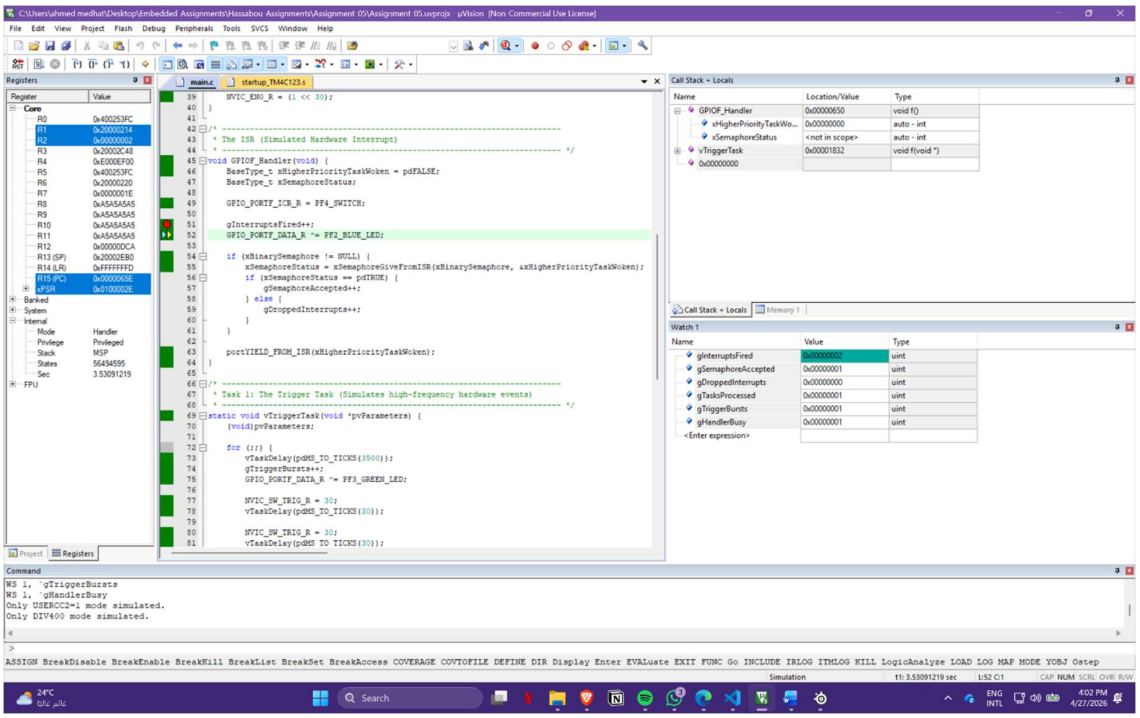

Figure 7 Second ISR Event Registration

Stepping over the ISR increment yields:

- gInterruptsFired = 2
- gTasksProcessed = 1
- gHandlerBusy = 1

This validates the core overload parameter required by the assignment: a secondary hardware event occurred precisely during the handler's blocked processing window.

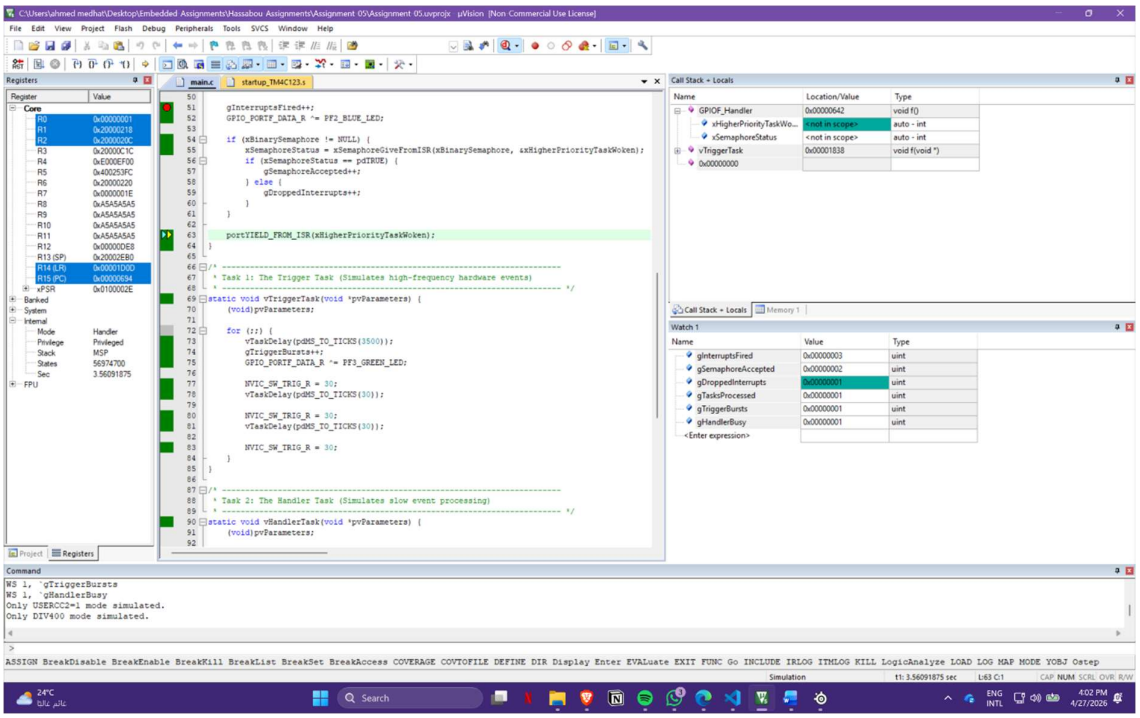

Figure 8 Definitive Proof of Event Loss

This final state provides the conclusive proof of the architectural bottleneck.

- gInterruptsFired = 3
- gSemaphoreAccepted = 2
- gDroppedInterrupts = 1
- gTasksProcessed = 1
- gHandlerBusy = 1

While three independent interrupts were generated, the binary semaphore could only successfully queue two tokens, resulting in a documented data loss (gDroppedInterrupts = 1). Because the processing task had only cleared a single event and remained busy, this empirically demonstrates the failure of a binary semaphore to buffer multiple pending events.

3.1 Experimental Result Summary

The observed execution sequence confirms the theoretical limitations of the binary semaphore behavior through five distinct phases:

1. The initial interrupt was accepted and successfully awakened the handler task.
2. The handler task transitioned to an artificially prolonged busy state to process the event.
3. Subsequent asynchronous interrupts arrived during this occupied processing window.
4. The binary semaphore, limited to a strictly dual-state architecture, failed to queue the supplementary incoming interrupts.
5. At least one interrupt was permanently discarded by the RTOS, validated physically by the gDroppedInterrupts = 1 register state.

4.0 Interpretation & Engineering Trade-offs

The data clearly illustrates the fundamental limitation of a binary semaphore in a real-time system. A binary semaphore does not possess an internal counter; it merely registers a binary pending state. Therefore, when multiple interrupts occur before the handler task can cycle back to a Blocked state waiting on xSemaphoreTake(), the RTOS has no mechanism to preserve the additional events.

5.0 Conclusion

The assignment objectives were successfully achieved. The implementation definitively proves that under overload conditions—where the ISR frequency exceeds the task execution rate—binary semaphores are an inadequate synchronization primitive and lead directly to dropped events. For systems requiring strict event preservation under such conditions, a Counting Semaphore or an RTOS Message Queue would be the required architectural upgrade.